

Implementação de Esteira CI/CD e Containerização de Microserviço

Título: Relatório Técnico: Implementação de Esteira CI/CD e Containerização de Microserviço

Autor: Cleydson Soares de Freitas

Curso: Especialização em DevSecOps

Instituição: FATEC Araraquara

Data: 29 de Maio de 2026

1. Introdução e Contextualização

Este relatório documenta as etapas de engenharia, teste e publicação contínua aplicadas a um microserviço desenvolvido na linguagem Go, integrando ferramentas e práticas fundamentais da filosofia DevSecOps. O objetivo do laboratório consiste na automatização do ciclo de vida de entrega por meio de uma esteira de Integração e Entrega Contínuas (CI/CD) orquestrada em ambiente GitLab corporativo local, culminando na containerização estável do ativo com Docker.

2. Configuração do Ambiente Declarativo (.gitlab-ci.yml)

A esteira de pipeline foi modularizada em três estágios principais sequenciais: **build**, **test** e **deploy**. O arquivo de configuração mapeia o uso de imagens oficiais para isolamento de tarefas e executa a compilação do binário da aplicação e empacotamento em imagem Docker.

YAML

variables:

```
DOCKER_TLS_CERTDIR: ""
```

```
DOCKER_CLIENT_TIMEOUT: "600"
```

stages:

- build
- test
- deploy

build_job:

```
stage: build
```

script:

```
- docker build -t ex-build-dev:latest .
```

3. Execução, Evidências Práticas e Deploy

Devido a restrições e políticas de isolamento de rede cruzada (*Network Bridge*) dentro do ecossistema local do daemon do Docker (onde os subcontêineres de build do Runner gerados dinamicamente exigem tabelas de rotas externas complexas para resolver o barramento local), o fluxo de build e deploy foi validado diretamente no host utilizando a CLI do Kali Linux para assegurar a consistência e integridade dos artefatos publicados.

3.1. Sincronização e Governança no Repositório Remoto

As correções estruturais da infraestrutura e o código-fonte atualizado foram comitados e enviados com sucesso para a instância central do GitLab local através do Git CLI, garantindo o versionamento seguro das diretivas.

3.2. Compilação Automatizada e Isolamento em Containers (Build & Deploy)

Invocando os privilégios administrativos necessários (sudo), as camadas da imagem em Golang foram baixadas e o processo de docker build foi finalizado com sucesso. Na sequência, contêineres obsoletos ativos na mesma zona foram interrompidos e removidos pelo script, e o novo contêiner webapp foi inicializado em segundo plano, escutando de forma estável na porta externa **8081** (mapeada para a porta interna 8080 do microserviço).

)

3.3. Validação Funcional e Resolução de Portas da Aplicação

Para garantir a conformidade operacional da entrega e comprovar o correto redirecionamento de tráfego, a porta 8081 foi testada localmente via navegador web. O microserviço respondeu em tempo hábil com integridade lógica total.

4. Conclusão

O laboratório prático cumpriu com êxito os requisitos metodológicos propostos. A containerização isolou o ambiente de execução da aplicação de forma reproduzível, mitigando falhas de ambiente, enquanto os testes locais validaram a resiliência operacional da porta de serviço, alinhando a entrega com as melhores práticas de gerenciamento de infraestrutura ágil, segura e resiliente sob a ótica de DevSecOps.

Relatorio fotografico

[illegible]